

AI-enabled IaaS Application

Best Practices To-Do List for 3,000 Initial Users and Easy Scale

Stack focus: MERN / MEAN-style development, Node.js backend, React frontend, MongoDB, Redis, AI agents, AWS integration, Terraform generation, and solo-developer scalability.

Executive Summary

Your app is not a normal CRUD product. It combines a SaaS dashboard, AI agent, AWS account integration, infrastructure diagram builder, Terraform generation, and potentially live infrastructure execution. Design it like a mini cloud platform from day one.

- Start with a modular monolith plus background workers, not full microservices.
- Separate heavy jobs such as AWS sync, AI analysis, and Terraform runs from API requests.
- Use strong multi-tenancy, RBAC, audit logs, rate limits, and isolated Terraform execution.
- Let AI assist and explain, but require human approval before creating, changing, or destroying infrastructure.
- Focus first on secure account connection, resource dashboard, diagram builder, Terraform plan, approval workflow, and then AI on top.

1. Recommended Initial Architecture

For 3,000 users, do not start with complex microservices. Start with a modular monolith plus separate worker processes. This is easier for a solo developer and still scales well.

```
React Frontend
|
API Gateway / Load Balancer
|
Node.js / NestJS Backend API
|
MongoDB Atlas
Redis Cache / Queue
Background Workers
|
AWS APIs / Terraform Engine / AI Agent / Billing Sync
```

Recommended Backend Modules

```
/src
/auth
/users
/organizations
/projects
/aws-accounts
/infra-designer
/terraform
/ai-agent
/billing
/audit-logs
/notifications
```

2. Frontend Checklist: React

Use React as a production SaaS frontend. For the n8n-like infrastructure diagram builder, use React Flow or a similar node-canvas library.

- React + TypeScript
- Vite or Next.js
- React Query / TanStack Query
- Zustand or Redux Toolkit
- React Hook Form + Zod
- Tailwind, ShadCN, or MUI
- React Flow for infrastructure diagram builder

Frontend Best Practices

- Lazy loading and route-based code splitting
- Error boundaries and global API error handling
- Role-based route guards
- Skeleton loaders
- Optimistic UI only where safe
- Refresh-token handling
- Dark/light mode
- Audit-friendly UX
- Separate diagram canvas state from global app state

```
/src
/app
/features
  /auth
  /dashboard
  /infra-builder
  /aws-accounts
  /billing
  /ai-agent
/components
/hooks
/services
/types
/utls
```

3. Backend Checklist: Node.js / NestJS

For a serious solo SaaS app, prefer NestJS over plain Express because it gives modules, dependency injection, guards, interceptors, pipes, exception filters, and better structure for a large backend.

- TypeScript everywhere
- DTO validation
- Centralized error handling
- Request logging
- Correlation ID
- Rate limiting
- JWT authentication
- Refresh-token rotation

- RBAC
- Multi-tenant organization model
- Audit logs
- Background jobs
- API versioning
- Swagger/OpenAPI

4. MongoDB Design Checklist

MongoDB Atlas is fine for your initial user target, but schema design and indexes matter.

```
users
organizations
organization_members
aws_accounts
projects
infra_diagrams
terraform_runs
ai_agent_sessions
billing_snapshots
cloud_resources
audit_logs
notifications
api_keys
```

Multi-tenant Fields

```
{
  organizationId: ObjectId,
  createdBy: ObjectId,
  createdAt: Date,
  updatedAt: Date
}
```

Never query user data without organizationId. This is one of the most important SaaS safety rules.

```
db.projects.find({
  organizationId: orgId,
  createdBy: userId
});
```

Index Checklist

- users: email unique
- projects: organizationId + createdAt
- infra_diagrams: projectId + organizationId
- terraform_runs: projectId + status + createdAt
- billing_snapshots: awsAccountId + date
- audit_logs: organizationId + createdAt
- cloud_resources: awsAccountId + resourceType + region

5. Scaling Target: 3,000 Users

Design for active and concurrent users, not only total registered users.

Total users: 3,000
 Daily active users: 500-1,000
 Concurrent users: 100-300
 Heavy concurrent infra operations: 20-50

The heaviest operations will be AI chat, AWS resource sync, Terraform plan/apply, diagram save/versioning, billing/resource scans, and audit logging.

Recommended Runtime Services

- API service
- Worker service
- Scheduler service
- AI agent service
- Terraform runner service
- Webhook service

```
npm run start:api
npm run start:worker
npm run start:scheduler
npm run start:terraform-runner
```

6. Background Job Checklist

Never run heavy AWS or Terraform work directly inside the API request.

Bad:
 User clicks Apply Infra -> API directly runs Terraform -> API waits -> timeout risk

Good:
 User clicks Apply Infra -> API creates job -> Worker runs job -> Status updated -> UI polls or receives WebSocket event

- Use BullMQ + Redis initially
- Move to AWS SQS later if needed
- Track status and logs for every job
- Add retries, backoff, timeout, and dead-letter handling

Suggested Job Types

- syncAwsResources
- syncBilling
- generateTerraform
- terraformPlan
- terraformApply
- terraformDestroy
- aiAccountAnalysis
- sendNotification

7. AI Agent Architecture

The AI agent should not directly perform dangerous actions. It should read, explain, suggest, and prepare plans. A user should approve infrastructure changes.

User question
 |

```

AI agent
|
Tool selection
|
Permission check
|
Read-only AWS tool / Billing tool / Resource tool
|
Response with grounded data

For write actions:
AI suggests action
User reviews
Terraform plan generated
User approves
Terraform apply runs
Audit log created

```

AI Agent Tools

- `getAwsBillingSummary`
- `listEC2Instances`
- `listS3Buckets`
- `getLambdaFunctions`
- `getRDSInstances`
- `getSecurityGroups`
- `generateTerraformFromDiagram`
- `explainTerraformPlan`
- `detectCostOptimization`
- `detectSecurityRisks`

Every Tool Should Have

- Input schema
- Output schema
- Permission check
- Rate limit
- Audit log
- Timeout
- Retry policy

8. Terraform Best Practices

```

Diagram JSON
-> Internal infra model
-> Terraform generator
-> Terraform plan
-> User approval
-> Terraform apply

```

Do not generate Terraform directly from UI JSON. First convert it into your own internal normalized model.

```

{
  projectId,
  provider: "aws",
  resources: [
    { id: "lambda_1", type: "aws_lambda_function", config: {} },

```

```
{ id: "api_gateway_1", type: "aws_api_gateway", config: {} }
],
connections: []
}
```

Terraform Safety Checklist

- Run Terraform in an isolated container
- Never run Terraform on the main API server
- Use remote state
- Use state locking
- Store logs and plan output
- Require approval before apply
- Require extra approval before destroy
- Limit IAM permissions
- Encrypt secrets
- Never expose tfstate to the frontend

9. AWS Integration Checklist

Use cross-account IAM role access, not permanent user access keys.

User creates IAM role in their AWS account
 Your app stores role ARN
 Backend assumes role using AWS STS
 Temporary credentials are used
 Credentials expire automatically

AWS Data Sync Strategy

- Billing: every 6-12 hours
- Resources: every 15-60 minutes
- Security checks: daily
- Manual refresh: on demand
- Do not call AWS APIs on every dashboard load; cache results in MongoDB/Redis

10. Security Checklist

Authentication

- JWT access token
- Refresh-token rotation
- HttpOnly secure cookies
- MFA later
- OAuth login later
- Session revoke
- Device/session tracking

Authorization / RBAC

- Owner
- Admin

- DevOps
- Developer
- Viewer
- Billing Viewer

Permissions

- project:create
- diagram:update
- terraform:plan
- terraform:apply
- terraform:destroy
- billing:view
- aws:sync
- users:invite

API Security

- Helmet
- CORS whitelist
- Rate limiting
- Input validation
- MongoDB injection protection
- XSS protection
- CSRF protection if using cookies
- Request size limit
- File upload validation
- Audit logs
- Secret scanning

Dependency Security

```
npm audit
npm outdated
 snyk test
depcheck
```

- Use package-lock.json
- Avoid unknown packages
- Pin important versions
- Review install scripts
- Do not blindly copy package README code
- Use GitHub Dependabot

11. Observability Checklist

- Structured logging
- Request ID / correlation ID
- Error tracking
- Metrics
- Tracing

- Health checks
- Audit logs
- Job status tracking
- Alerting

Recommended Stack

- Logs: CloudWatch or Grafana Loki
- Metrics: Prometheus + Grafana
- Errors: Sentry
- Tracing: OpenTelemetry
- Uptime: Better Stack / UptimeRobot

Metrics to Track

- API latency
- Error rate
- Login failures
- AI token usage
- AWS API failures
- Terraform job duration
- Queue length
- MongoDB slow queries
- Redis memory usage
- Worker failures
- Cost per organization

12. Deployment Checklist

Stage 1: Simple Production

- Frontend: Vercel or S3 + CloudFront
- Backend: AWS ECS Fargate, Render, Railway, or EC2
- Database: MongoDB Atlas
- Redis: Upstash or ElastiCache
- Storage: S3
- Queue: Redis BullMQ

Stage 2: Scalable Production

- Frontend: CloudFront + S3
- Backend: ECS Fargate
- Workers: ECS Fargate
- Redis: ElastiCache
- MongoDB: Atlas M10/M20+
- Secrets: AWS Secrets Manager
- Logs: CloudWatch
- CI/CD: GitHub Actions

Stage 3: Advanced Production

- Kubernetes / EKS
- Horizontal Pod Autoscaler
- KEDA for queue-based scaling
- Service mesh if needed
- Multi-region read strategy
- Advanced observability

13. CI/CD Checklist

- Install dependencies
- Lint
- Type check
- Unit tests
- Integration tests
- Build
- Docker image build
- Security scan
- Deploy to staging
- Manual approval
- Deploy to production

Branching

main -> production
develop -> staging
feature/* -> pull request

Quality Gates

- No direct push to main
- PR review required
- Tests required
- Secrets scanning
- ESLint required
- Prettier required

14. Performance Checklist

API

- Pagination everywhere
- No unbounded queries
- Use projection in MongoDB
- Cache dashboard data
- Use compression
- Avoid synchronous heavy work
- Use worker queues
- Use connection pooling

MongoDB

- Indexes for every common filter
- Avoid huge documents
- Avoid unlimited embedded arrays
- Use aggregation carefully
- Use explain plans
- Archive old logs

Redis Cache Examples

- AWS resource summary
- Billing summary
- User permissions
- Dashboard counts
- AI conversation metadata
- Terraform template catalog

Do not cache secrets, temporary AWS credentials, sensitive tfstate data, or unapproved Terraform plans forever.

15. Cost-control Checklist

- AI token cost
- AWS API call cost
- Worker compute cost
- Database cost
- Redis cost
- Storage cost
- Logging cost

AI Cost Controls

- Use smaller models for simple tasks
- Cache repeated analysis
- Summarize old chats
- Limit context size
- Use retrieval only when needed
- Set max token limits
- Track usage per organization

16. Infra Builder Data Model

- infra_diagrams
- diagram_versions
- terraform_runs
- terraform_states

```
{
  projectId,
  version: 12,
  nodes: [],
```

```
edges: [],
normalizedInfraModel: {},
createdBy,
createdAt
}
```

Versioning allows rollback, exact Terraform plan mapping, better audit, and easier debugging.

17. Audit Log Checklist

- User login
- AWS account connected
- Diagram created/updated/deleted
- Terraform generated
- Terraform plan created
- Terraform apply approved
- Terraform destroy requested
- Billing viewed
- AI agent action performed
- Role changed
- User invited/removed

```
{
  organizationId,
  actorUserId,
  action: "terraform.apply.approved",
  resourceType: "project",
  resourceId: projectId,
  ipAddress,
  userAgent,
  metadata: {},
  createdAt
}
```

18. Advanced Concepts to Learn as a Solo Developer

Phase 1: Production MERN Architecture

- Advanced TypeScript
- NestJS
- Clean Architecture in Node.js
- DTO validation
- RBAC
- Multi-tenancy
- MongoDB indexing
- Redis caching
- BullMQ workers
- WebSockets
- API rate limiting

Phase 2: Cloud + DevOps

- Docker

- Docker Compose
- GitHub Actions
- AWS ECS Fargate
- S3
- CloudFront
- Secrets Manager
- CloudWatch
- IAM
- STS AssumeRole
- Terraform basics
- Terraform state

Phase 3: AI Engineering

- Prompt engineering
- Function calling/tool calling
- RAG
- Embeddings
- Vector DB
- AI guardrails
- AI evaluation
- Token optimization
- Model fallback
- Agent workflows

Phase 4: Platform Engineering

- Kubernetes basics
- Helm
- Ingress
- HPA
- KEDA
- OpenTelemetry
- Prometheus
- Grafana
- SLO/SLI
- Load testing
- Chaos testing basics

19. Immediate 30-day To-do List

Week 1: Foundation

- Set up MERN/NestJS monorepo
- Add TypeScript
- Add authentication
- Add organization/team model
- Add RBAC
- Add MongoDB indexes

- Add audit logs
- Add Swagger

Week 2: Infra Builder

- Create React Flow canvas
- Create AWS resource node types
- Save diagram JSON
- Create diagram versioning
- Create normalized infra model
- Generate basic Terraform code

Week 3: AWS Integration

- Implement AWS account connection using IAM role ARN
- Use STS AssumeRole
- Fetch EC2/S3/Lambda/RDS resources
- Store resource snapshot
- Show dashboard
- Add billing summary

Week 4: AI + Jobs

- Add BullMQ worker
- Add AI agent service
- Create read-only AWS tools
- Create billing analysis tool
- Create Terraform explanation tool
- Add job status page
- Add logs page

20. Minimum Production Stack Recommendation

Frontend: React + TypeScript + React Flow + Tailwind
 Backend: NestJS + TypeScript
 Database: MongoDB Atlas
 Cache/Queue: Redis + BullMQ
 AI: OpenAI / Bedrock / Gemini later
 Infra generation: Terraform CLI in isolated worker
 Deployment: AWS ECS Fargate
 Storage: S3
 Secrets: AWS Secrets Manager
 Monitoring: Sentry + CloudWatch + OpenTelemetry
 CI/CD: GitHub Actions

Most Important Rule

Secure cloud account connection
 -> Resource dashboard
 -> Diagram builder
 -> Terraform generation
 -> Manual approval workflow
 -> AI assistant on top

AI should assist the user, not become the uncontrolled core controller of infrastructure. That is how your app becomes reliable, safe, and scalable.